arXiv:2406.18009v2 [eess.AS] 12 Sep 2024

E2 TTS: EMBARRASSINGLY EASY FULLY NON-AUTOREGRESSIVE ZERO-SHOT TTS

Sefik Emre Eskimez, Xiaofei Wang, Manthan Thakker, Canrun Li, Chung-Hsien Tsai, Zhen Xiao, Hemin Yang, Zirun Zhu, Min Tang, Xu Tan, Yanqing Liu, Sheng Zhao, Naoyuki Kanda

Microsoft Corporation, USA

ABSTRACT

This paper introduces Embarrassingly Easy Text-to-Speech (E2 TTS), a fully non-autoregressive zero-shot text-to-speech system that offers human-level naturalness and state-of-the-art speaker similarity and intelligibility. In the E2 TTS framework, the text input is converted into a character sequence with filler tokens. The flow-matching-based mel spectrogram generator is then trained based on the audio infilling task. Unlike many previous works, it does not require additional components (e.g., duration model, grapheme-to-phoneme) or complex techniques (e.g., monotonic alignment search). Despite its simplicity, E2 TTS achieves state-of-the-art zero-shot TTS capabilities that are comparable to or surpass previous works, including Voicebox and NaturalSpeech 3. The simplicity of E2 TTS also allows for flexibility in the input representation. We propose several variants of E2 TTS to improve usability during inference. See `https://aka.ms/e2tts/` for demo samples.

***Index Terms*—** zero-shot text-to-speech, flow-matching

1. INTRODUCTION

In recent years, text-to-speech (TTS) systems have seen significant improvements [1, 2, 3, 4], achieving a level of naturalness that is indistinguishable from human speech [5]. This advancement has further led to research efforts to generate natural speech for any speaker from a short audio sample, often referred to as an audio prompt. Early studies of zero-shot TTS used speaker embedding to condition the TTS system [6, 7]. More recently, VALL-E [8] proposed formulating the zero-shot TTS problem as a language modeling problem in the neural codec domain, achieving significantly improved speaker similarity while maintaining a simplistic model architecture. Various extensions were proposed to improve stability [9, 10, 11], and VALL-E 2 [12] recently achieved human-level zero-shot TTS with techniques including repetition-aware sampling and grouped code modeling.

While the neural codec language model-based zero-shot TTS achieved promising results, there are still a few limitations based on its auto-regressive (AR) model-based architecture. Firstly, because the codec token needs to be sampled sequentially, it inevitably increases the inference latency. Secondly, a dedicated effort to figure out the best tokenizer (both for text tokens and audio tokens) is necessary to achieve the best quality [9]. Thirdly, it is required to use some tricks to stably handle long sequences of audio codecs, such as the combination of AR and non-autoregressive (NAR) modeling [8, 12], multi-scale transformer [13], grouped code modeling [12].

Meanwhile, several fully NAR zero-shot TTS models have been proposed with promising results. Unlike AR-based models, fully NAR models enjoy fast inference based on parallel processing. NaturalSpeech 2 [14] and NaturalSpeech 3 [15] estimate the latent vectors of a neural audio codec based on diffusion models [16, 17]. Voicebox [18] and Matcha-TTS [19] used a flow-matching model [20] conditioned by an input text. However, one notable challenge for such NAR models is how to obtain the alignment between the input text and the output audio, whose length is significantly different. NaturalSpeech 2, NaturalSpeech 3, and Voicebox used a frame-wise phoneme alignment for training the model. Matcha-TTS, on the other hand, used monotonic alignment search (MAS) [21, 3] to automatically find the alignment between input and output. While MAS could alleviate the necessity of the frame-wise phoneme aligner, it still requires an independent duration model to estimate the duration of each phoneme during inference. More recently, E3 TTS [22] proposed using cross-attention from the input sequence, which required a carefully designed U-Net architecture [23]. As such, fully NAR zero-shot TTS models require either an explicit duration model or a carefully designed architecture. One of our findings in this paper is that such techniques are not necessary to achieve high-quality zero-shot TTS, and they are sometimes even harmful to naturalness.[1]

Another complexity in TTS systems is the choice of the text tokenizer. As discussed above, the AR-model-based system requires a careful selection of tokenizer to achieve the best result. On the other hand, most fully NAR models assume a monotonic alignment between text and output, with the exception of E3 TTS, which uses cross-attention. These models impose constraints on the input format and often require a text normalizer to avoid invalid input formats. When the model is trained based on phonemes, a grapheme-to-phoneme converter is additionally required.

In this paper, we propose **Embarrassingly Easy TTS (E2 TTS)**, a fully NAR zero-shot TTS system with a surprisingly simple architecture. E2 TTS consists of only two modules: a flow-matching-based mel spectrogram generator and a vocoder. The text input is converted into a character sequence with filler tokens to match the length of the input character sequence and the output mel-filterbank sequence. The mel spectrogram generator, composed of a vanilla Transformer with U-Net style skip connections, is trained using a speech-infilling task [18]. Despite its simplicity, E2 TTS achieves state-of-the-art zero-shot TTS capabilities that are comparable to, or surpass, previous works, including Voicebox and NaturalSpeech 3. The simplicity of E2 TTS also allows for flexibility in the input representation. We propose several variants of E2 TTS to improve usability during inference.

2. E2 TTS

2.1. Training

Fig. 1 (a) provides an overview of E2 TTS training. Suppose we have a training audio sample s with transcription $y = (c_1, c_2, ..., c_M)$,

[1] Concurrent with our work, Seed-TTS [24] proposed a diffusion model-based zero-shot TTS, named Seed-TTS$_{\mathrm{DiT}}$. Although it appears to share many similarities with our approach, the authors did not elaborate on the details of their model, making it challenging to compare with our work.

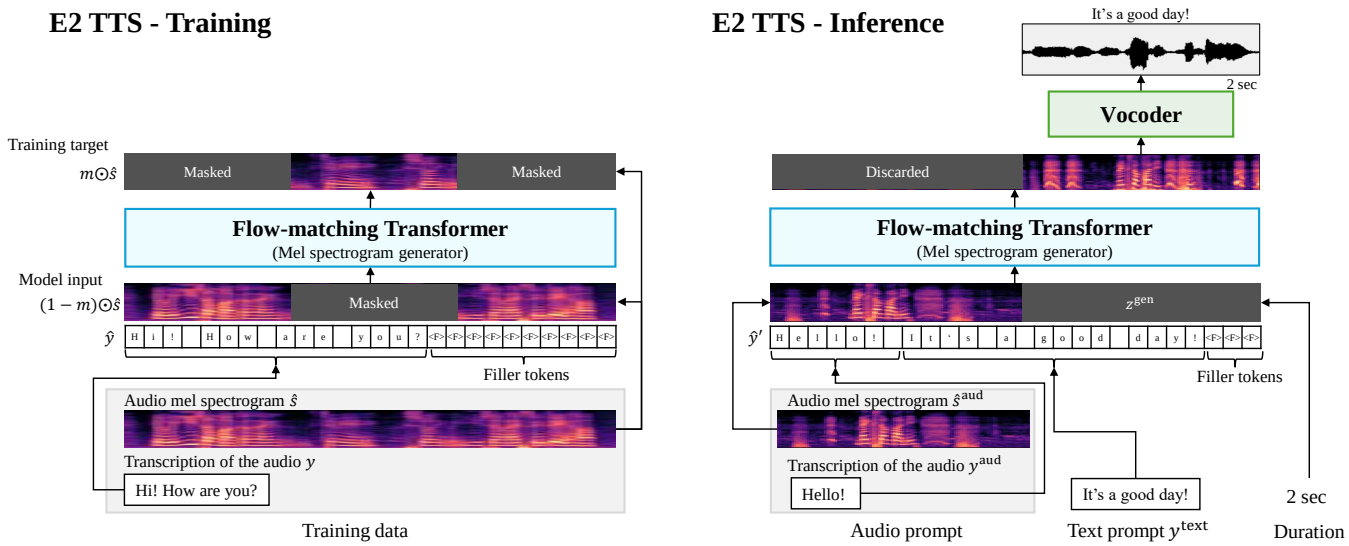


Fig. 1. An overview of the training (left) and the inference (right) processes of E2 TTS.

where c_i represents the i-th character of the transcription.[2] First, we extract its mel-filterbank features $\hat{s} \in \mathbb{R}^{D \times T}$, where D denotes the feature dimension and T represents the sequence length. We then create an extended character sequence $\hat{y}$, where a special filler token $\langle F \rangle$ is appended to y to make the length of $\hat{y}$ equal to T.[3]

$$\hat{y} = (c_1, c_2, \ldots, c_M, \underbrace{\langle F \rangle, \ldots, \langle F \rangle}_{(T-M) \text{ times}}). \tag{1}$$

A spectrogram generator, consisting of a vanilla Transformer [26] with U-net [23] style skip connection, is then trained based on the speech infilling task [18]. More specifically, the model is trained to learn the distribution $P(m \odot \hat{s} | (1-m) \odot \hat{s}, \hat{y})$, where $m \in \{0,1\}^{D \times T}$ represents a binary temporal mask, and $\odot$ is the Hadamard product. E2 TTS uses the conditional flow-matching [20] to learn such distribution.

2.2. Inference

Fig. 1 (b) provides an overview of the inference with E2 TTS. Suppose we have an audio prompt s^{aud} and its transcription $y^{\text{aud}} = (c'_1, c'_2, ..., c'_{M^{\text{aud}}})$ to mimic the speaker characteristics. We also suppose a text prompt $y^{\text{text}} = (c''_1, c''_2, ..., c''_{M^{\text{text}}})$. In the E2 TTS framework, we also require the target duration of the speech that we want to generate, which may be determined arbitrarily. The target duration is internally represented by the frame length T^{gen}.

First, we extract the mel-filterbank features $\hat{s}^{\text{aud}} \in \mathbb{R}^{D \times T^{\text{aud}}}$ from s^{aud}. We then create an extended character sequence $\hat{y}'$ by concatenating y^{aud}, y^{text}, and repeated $\langle F \rangle$, as follows:

$$\hat{y}' = (c'_1, c'_2, \ldots, c'_{M^{\text{aud}}}, c''_1, c''_2, \ldots, c''_{M^{\text{text}}}, \underbrace{\langle F \rangle, \ldots, \langle F \rangle}_{\mathcal{T} \text{ times}}), \tag{2}$$

where $\mathcal{T} = T^{\text{aud}} + T^{\text{gen}} - M^{\text{aud}} - M^{\text{text}}$, which ensures the length of $\hat{y}'$ is equal to $T^{\text{aud}} + T^{\text{gen}}$.[4]

[2] Alternatively, we can represent y as a sequence of Unicode bytes [25].

[3] We assume $M \leq T$, which is almost always valid.

[4] To ensure $\mathcal{T} \geq 0$, T^{gen} needs to satisfy $T^{\text{gen}} \geq M^{\text{aud}} + M^{\text{text}} - T^{\text{aud}}$, which is almost always valid in the TTS scenario.

The mel spectrogram generator then generates mel-filterbank features $\tilde{s}$ based on the learned distribution of $P(\tilde{s}|[\hat{s}^{\text{aud}}; z^{\text{gen}}], \hat{y}')$, where z^{gen} is an all-zero matrix with a shape of $D \times T^{\text{gen}}$, and $[;]$ is a concatenation operation in the dimension of T^*. The generated part of $\tilde{s}$ are then converted to the speech signal based on the vocoder.

2.3. Flow-matching-based mel spectrogram generator

E2 TTS leverages conditional flow-matching [20], which incorporates the principles of continuous normalizing flows [27]. This model operates by transforming a simple initial distribution p_0 into a complex target distribution p_1 that characterizes the data. The transformation process is facilitated by a neural network, parameterized by θ, which is trained to estimate a time-dependent vector field, denoted as $v_t(x; \theta)$, for $t \in [0, 1]$. From this vector field, we derive a flow, ϕ_t, which effectively transitions p_0 into p_1. The neural network's training is driven by the conditional flow matching objective:

$$\mathcal{L}^{\text{CFM}}(\theta) = \mathbb{E}_{t, q(x_1), p_t(x|x_1)} \left\| u_t(x|x_1) - v_t(x; \theta) \right\|^2, \tag{3}$$

where p_t is the probability path at time t, u_t is the designated vector field for p_t, x_1 symbolizes the random variable corresponding to the training data, and q is the distribution of the training data. In the training phase, we construct both a probability path and a vector field from the training data, utilizing an optimal transport path: $p_t(x|x_1) = \mathcal{N}(x|tx_1, (1-(1-\sigma_{\min})t)^2 I)$ and $u_t(x|x_1) = (x_1 - (1-\sigma_{\min})x)/(1-(1-\sigma_{\min})t)$. For inference, we apply an ordinary differential equation (ODE) solver [27] to generate the log mel-filterbank features starting from the initial distribution p_0.

We adopt the same model architecture with the audio model of Voicebox (Fig. 2 of [18]) except that the frame-wise phoneme sequence is replaced into $\hat{y}$. Specifically, Transformer with U-Net style skip connection [18] is used as a backbone. The input to the mel spectrogram generator is $(1-m) \odot \hat{s}$, $\hat{y}$, the flow step t, and noisy speech s_t. $\hat{y}$ is first converted to character embedding sequence $\tilde{y} \in \mathbb{R}^{E \times T}$. Then, $(1-m) \odot \hat{s}$, s_t, $\tilde{y}$ are all stacked to form a tensor with a shape of $(2 \cdot D + E) \times T$, followed by a linear layer to output a tensor with a shape of $D \times T$. Finally, an embedding representation, $\hat{t} \in \mathbb{R}^D$, of t is appended to form the input tensor with a shape of $\mathbb{R}^{D \times (T+1)}$ to the Transformer. The Transformer is

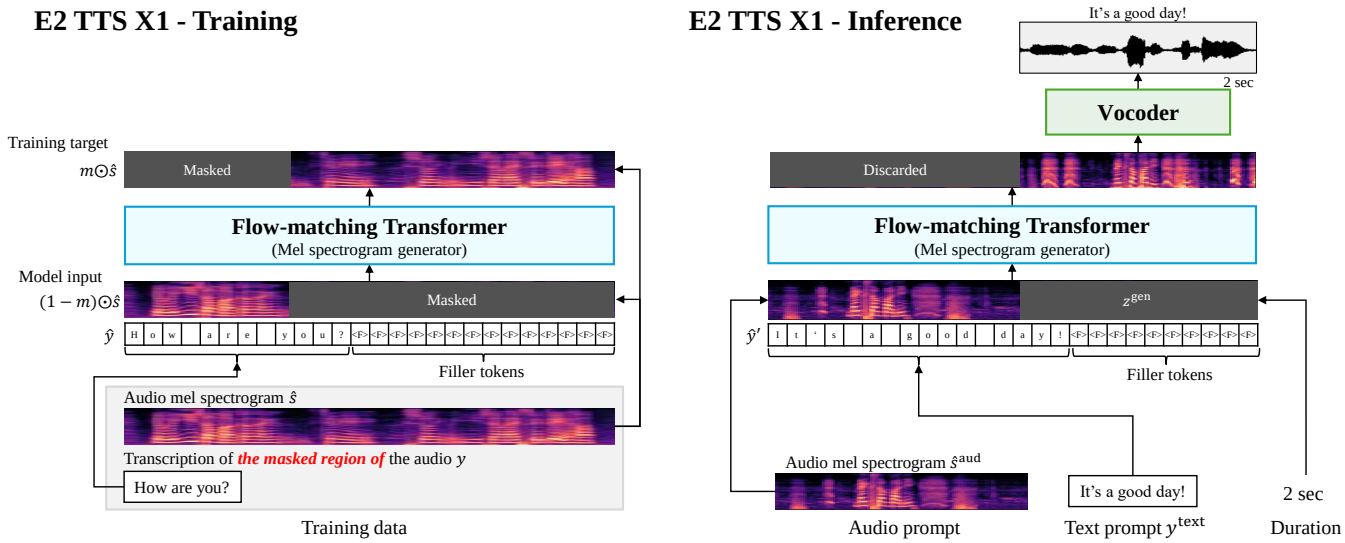


Fig. 2. An overview of the training (left) and the inference (right) processes of E2 TTS X1.

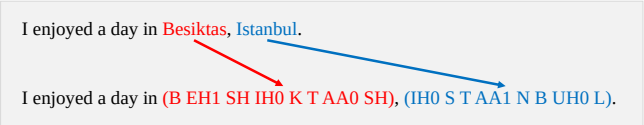


Fig. 3. Example of the transcription for E2 TTS X2 where words are replaced with phoneme sequences enclosed in parentheses.

trained to output a vector field v_t with the conditional flow-matching objective $\mathcal{L}^{\text{CFM}}$.

2.4. Relationship to Voicebox

E2 TTS has a close relationship with the Voicebox. From the perspective of the Voicebox, E2 TTS replaces a frame-wise phoneme sequence used in conditioning with a character sequence that includes a filler token. This change significantly simplifies the model by eliminating the need for a grapheme-to-phoneme converter, a phoneme aligner, and a phoneme duration model. From another viewpoint, the mel spectrogram generator of E2 TTS can be viewed as a joint model of the grapheme-to-phoneme converter, the phoneme duration model, and the audio model of the Voicebox. This joint modeling significantly improves naturalness while maintaining speaker similarity and intelligibility, as will be demonstrated in our experiments.

2.5. Extension of E2 TTS

2.5.1. Extension 1: Eliminating the need for transcription of audio prompts in inference

In certain application contexts, obtaining the transcription of the audio prompt can be challenging. To eliminate the requirement of the transcription of the audio prompt during inference, we introduce an extension, illustrated in Fig. 2. This extension, referred to as E2 TTS X1, assumes that we have access to the transcription of the masked region of the audio, which we use for y. During inference, the extended character sequence $\hat{y}'$ is formed without y^{aud}, namely,

$$\hat{y}' = (c''_1, c''_2, \ldots, c''_{M^{\text{text}}}, \underbrace{\langle F \rangle, \ldots, \langle F \rangle}_{\mathcal{T} \text{ times}}). \tag{4}$$

The rest of the procedure remains the same as in the basic E2 TTS.

The transcription of the masked region of the training audio can be obtained in several ways. One method is to simply apply automatic speech recognition (ASR) to the masked region during training, which is straightforward but costly. In our experiment, we employed the Montreal Forced Aligner [28] to determine the start and end times of words within each training data sample. The masked region was determined in such a way that we ensured not to cut the word in the middle.

2.5.2. Extension 2: Enabling explicit indication of pronunciation for parts of words in a sentence

In certain scenarios, users want to specify the pronunciation of a specific word such as unique foreign names. Retraining the model to accommodate such new words is both expensive and time-consuming.

To tackle this challenge, we introduce another extension that enables us to indicate the pronunciation of a word during inference. In this extension, referred to as E2 TTS X2, we occasionally substitute a word in y with a phoneme sequence enclosed in parentheses during training, as depicted in Fig. 3. In our implementation, we replaced the word in y with the phoneme sequence from the CMU pronouncing dictionary [29] with a 15% probability. During inference, we simply replace the target word with phoneme sequences enclosed in parentheses.

It's important to note that y is still a simple sequence of characters, and whether the character represents a word or a phoneme is solely determined by the existence of parentheses and their content. It's also noteworthy that punctuation marks surrounding the word are retained during replacement, which allows the model to utilize these punctuation marks even when the word is replaced with phoneme sequences.

3. EXPERIMENTS

3.1. Training data

We utilized the Libriheavy dataset [30] to train our models. The Libriheavy dataset comprises 50,000 hours of read English speech from 6,736 speakers, accompanied by transcriptions that preserve case and punctuation marks. It is derived from the Librilight [31] dataset contains 60,000 hours of read English speech from over 7,000 speakers. For E2 TTS training, we used the case and punctuated transcription without any pre-processing. We also used a proprietary 200,000

hours of training data to investigate the scalability of the E2 TTS model.

3.2. Model configurations

We constructed our proposed E2 TTS models using a Transformer architecture. The architecture incorporated U-Net [23] style skip connections, 24 layers, 16 attention heads, an embedding dimension of 1024, a linear layer dimension of 4096. The character embedding vocabulary size was 399.[5] The total number of parameters amounted to 335 million. We modeled the 100-dimensional log mel-filterbank features, extracted every 10.7 milliseconds from audio samples with a 24 kHz sampling rate. A BigVGAN [32]-based vocoder was employed to convert the log mel-filterbank features into waveforms. The masking length was randomly determined to be between 70% and 100% of the log mel-filterbank feature length during training. In addition, we randomly dropped all the conditioning information with a 20% probability for classifier-free guidance (CFG) [33]. All models were trained for 800,000 mini-batch updates with an effective mini-batch size of 307,200 audio frames. We utilized a linear decay learning rate schedule with a peak learning rate of 7.5×10^{-5} and incorporated a warm-up phase for the initial 20,000 updates. We discarded the training samples that exceeded 4,000 frames.

In a subset of our experiments, we initialized E2 TTS models using a pre-trained model in an unsupervised manner. This pre-training was conducted on an anonymized dataset, which consisted of 200,000 hours of unlabeled data. The pre-training protocol, which involved 800,000 mini-batch updates, followed the scheme outlined in [34].

In addition, we trained a regression-based duration model by following that of Voicebox [18]. It is based on a Transformer architecture, consisting of 8 layers, 8 attention heads, an embedding dimension of 512, a linear layer dimension of 2048. The training process involved 75,000 mini-batch updates with 120,000 frames. We used this duration model to estimate the target duration for a fair comparison with the Voicebox baseline. Note that we will also show that E2 TTS is robust for different target duration in Section 3.6.

3.3. Evaluation data and metrics

In order to assess our models, we utilized the test-clean subset of the LibriSpeech-PC dataset [35], which is an extension of LibriSpeech [36] that includes additional punctuation marks and casing. We specifically filtered the samples to retain only those with a duration between 4 and 10 seconds. Since LibriSpeech-PC lacks some of the utterances from LibriSpeech, the total number of samples was reduced to 1,132, sourced from 39 speakers. For the audio prompt, we extracted the last three seconds from a randomly sampled speech file from the same speaker for each evaluation sample.[6]

We carried out both objective and subjective evaluations. For the objective evaluations, we generated samples using three random seeds, computed the objective metrics for each, and then calculated their average. We computed the word error rate (WER) and speaker similarity (SIM-o). The WER is indicative of the intelligibility of the generated samples, and for its calculation, we utilized a Hubert-large-based [37] ASR system. The SIM-o represents the speaker similarity between the audio prompt and the generated sample, which is estimated by computing the cosine similarity between the speaker embeddings of both. For the calculation of SIM-o, we used a WavLM-large-based [38] speaker verification model.

[5] We used all characters and symbols that we found in the training data without filtering.

[6] The test set used in our experiments can be accessed at: `https://github.com/microsoft/e2tts-test-suite`.

Table 1. Objective results for LibriSpeech-PC test-clean evaluation set. WER is expressed in percentage. † Our reproduction. LL, LH, and PP stand for Librilight, Libriheavy, and Proprietary, respectively.

ID	Model	Data (hours)	Init	WER↓	SIM-o↑
(GT)	Ground Truth	-	-	2.0	0.695
(B1)	VALL-E [8]	LL (60K)	Random	4.9	0.500
(B2)	NaturalSpeech 3 [15]	LL (60K)	Random	2.6	0.632
(B3)	Voicebox [18]†	LL (60K)	Random	2.1	0.658
(B4)	Voicebox [18]†	LH (50K)	Random	2.2	0.667
(B5)	Voicebox [18]†	LH (50K)	Pretrain [34]	2.2	0.695
(P1)	E2 TTS	LH (50K)	Random	2.0	0.675
(P2)	E2 TTS	LH (50K)	Pretrain [34]	**1.9**	**0.708**
(P3)	E2 TTS	PP (200K)	Random	**1.9**	0.707

Table 2. Subjective results for LibriSpeech-PC test-clean evaluation set. † Our reproduction.

ID	Model	CMOS↑	SMOS↑
(GT)	Ground Truth	0.00	$3.91_{\pm 0.13}$
(B2)	NaturalSpeech 3 [15]	-0.98	$\mathbf{4.76}_{\pm 0.06}$
(B4)	Voicebox [18]†	-0.78	$4.73_{\pm 0.06}$
(P1)	E2 TTS	-0.14	$4.66_{\pm 0.07}$
(P2)	E2 TTS	**-0.05**	$4.65_{\pm 0.08}$
(P3)	E2 TTS	-0.18	$4.64_{\pm 0.08}$

For the subjective assessments, we conducted two tests: the Comparative Mean Opinion Score (CMOS) and the Speaker Similarity Mean Opinion Score (SMOS). We evaluated 39 samples for both tests, with one sample per speaker from our test-clean set. Each sample was assessed by 12 Native English evaluators. In the CMOS test [15], evaluators were shown the ground-truth sample and the generated sample side-by-side without knowing which was the ground-truth, and were asked to rate the naturalness on a 7-point scale (from -3 to 3), where a negative value indicates a preference for the ground-truth and a positive value indicates the opposite. In the SMOS test, evaluators were presented with the audio prompt and the generated sample, and asked to rate the speaker similarity on a scale of 1 (not similar at all) to 5 (identical), with increments of 1.

3.4. Main results

In our experiments, we conducted a comparison between our E2 TTS models and three other models: Voicebox [18], VALL-E [8], and NaturalSpeech 3 [15]. We utilized our own reimplementation of the Voicebox model, which was based on the same model configuration with E2 TTS except that the Vicebox model is trained with frame-wise phoneme alignment. During the inference, we used CFG with a guidance strength of 1.0 for both E2 TTS and Voicebox. We employed the midpoint ODE solver with a number of function evaluations of 32.

Table 1 presents the objective evaluation results for the baseline and E2 TTS models across various configurations. By comparing the (B4) and (P1) systems, we observe that the E2 TTS model achieved better WER and SIM-o than the Voicebox model when both were trained on the Libriheavy dataset. This trend holds even when we initialize the model with unsupervised pre-training [34] ((B5) vs. (P2)), where the (P2) system achieved the best WER (1.9%) and SIM-o (0.708) which are better than those of the ground-truth audio. Finally, by using larger training data (P3), E2 TTS achieved the same best WER (1.9%) and the second best SIM-o (0.707) even when the model is trained from scratch, showcasing the scalability of E2 TTS.

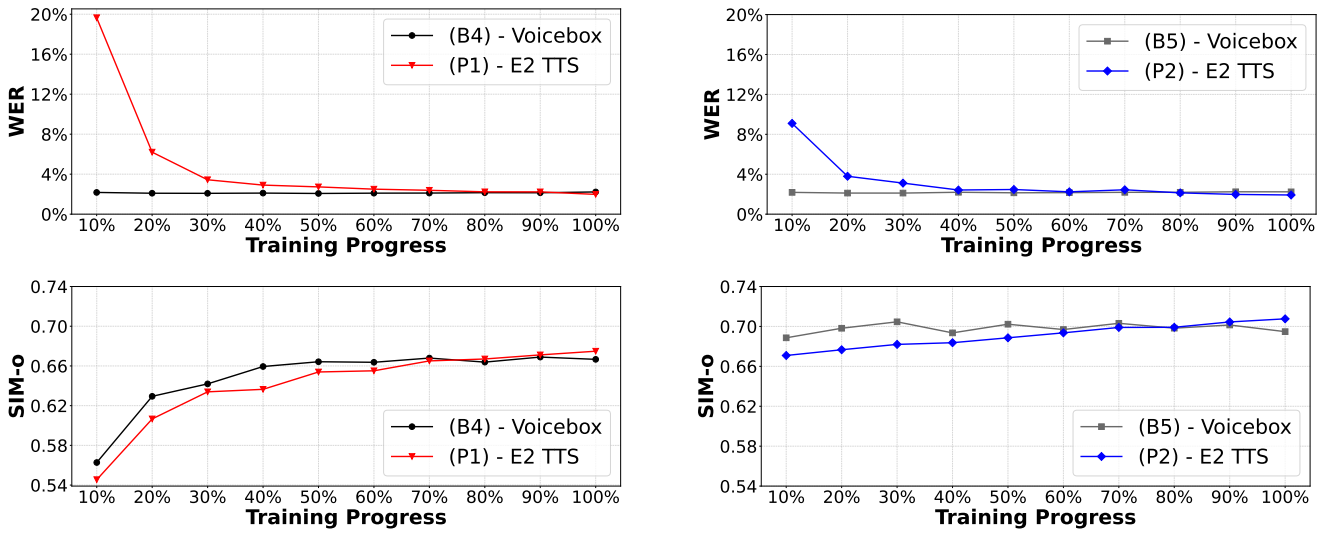


Fig. 4. The training progress of Voicebox and E2TTS models. The top row shows WER and the bottom row shows SIM progressions.

Table 3. Comparison between the basic E2 TTS and E2 TTS X1. While the basic E2 TTS requires the transcription of the audio prompt during inference, E2 TTS X1 does not require it. WER is expressed in percentage.

ID	Model	Init	WER↓	SIM-o↑
(P1)	E2 TTS	Random	2.0	0.675
(P2)	E2 TTS	Pre-trained [34]	1.9	0.708
(P1-X1)	E2 TTS X1	Random	2.0	0.664
(P2-X1)	E2 TTS X1	Pre-trained [34]	2.0	0.705

It is noteworthy that E2 TTS achieved superior performance compared to all strong baselines, including VALL-E, NaturalSpeech 3, and Voicebox, despite its extremely simple framework.

Table 2 illustrated the subjective evaluation results for NaturalSpeech 3, Voicebox, and E2 TTS. Firstly, all variants of E2 TTS, from (P1) to (P3), showed a better CMOS score compared to NaturalSpeech 3 and Voicebox. In particular, the (P2) model achieved a CMOS score of -0.05, which is considered to have a level of naturalness indistinguishable from the ground truth [15, 24].[7] The comparison between (B4) and (P1) suggests that the use of phoneme alignment was the major bottleneck in achieving better naturalness. Regarding speaker similarity, all the models we tested showed a better SMOS compared to the ground truth, a phenomenon also observed in NaturalSpeech 3 [15].[8] Among the tested systems, E2 TTS achieved a comparable SMOS to Voicebox and NaturalSpeech 3.

Overall, E2 TTS demonstrated a robust zero-shot TTS capability that is either superior or comparable to strong baselines, including Voicebox and NaturalSpeech 3. The comparison between Voicebox and E2 TTS revealed that the use of phoneme alignment was the primary obstacle in achieving natural-sounding audio. With a simple training scheme, E2 TTS can be easily scaled up to accommodate large training data. This resulted in human-level speaker similarity, intelligibility, and a level of naturalness that is indistinguishable from a human's voice in the zero-shot TTS setting, despite the framework's extreme simplicity.

[7] In the evaluation, E2 TTS was judged to be better in 33% of samples, while the ground truth was better in another 33% of samples. The remaining samples were judged to be of equal quality.

[8] In LibriSpeech, some speakers utilized varying voice characteristics for different characters in the book, leading to a low SMOS for the ground truth.

Table 4. The WER (%) and SIM-o of E2 TTS X2 where a word is randomly replaced with the phoneme sequence during inference. Even when we replaced 50% of words into phoneme sequences, E2 TTS X2 worked reasonably well. This indicates that we can specify the pronunciation of a new term without retraining.

ID	Model	Init	Phoneme %	WER↓	SIM-o↑
(P1)	E2 TTS	Random	0%	2.0	0.675
(P1-X2)	E2 TTS X2	Random	0%	2.0	0.679
			25%	2.0	0.678
			50%	2.1	0.679
(P2)	E2 TTS	Pre-trained [34]	0%	1.9	0.708
(P2-X2)	E2 TTS X2	Pre-trained [34]	0%	1.9	0.708
			25%	2.0	0.708
			50%	2.1	0.707

3.5. Evaluation of E2 TTS extensions

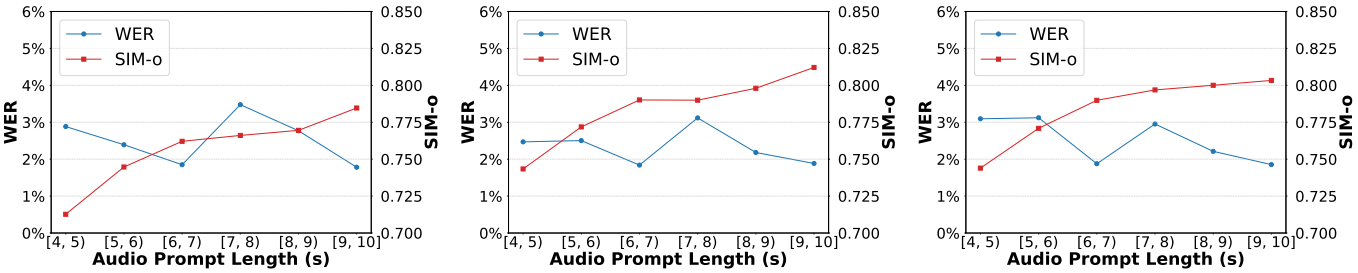
3.5.1. Evaluation of the extension 1

The results for the E2 TTS X1 models are shown in Table 3. These results indicate that the E2 TTS X1 model has achieved results nearly identical to those of the E2 TTS model, especially when the model was initialized by unsupervised pre-training [34]. E2 TTS X1 does not require the transcription of the audio prompt, which greatly enhances its usability.

3.5.2. Evaluation of the extension 2

In this experiment, we trained the E2 TTS X2 models with a phoneme replacement rate of 15%. During inference, we randomly replaced words in the test-clean dataset with phoneme sequences, with a probability ranging from 0% to 50%.

Table 4 shows the result. We first observed that E2 TTS X2 achieved parity results when no words were replaced with phoneme sequences. This shows that we can introduce extension 2 without any drawbacks. We also observed that the E2 TTS X2 model showed only marginal degradation of WER, even when we replaced 50% of words with phoneme sequences. This result indicates that we can specify the pronunciation of a new term without retraining the E2 TTS model.



(P1) E2 TTS (P2) E2 TTS (P3) E2 TTS

Fig. 5. The results of WER and SIM, shown in buckets according to the audio prompt length. The left, middle, and right plots show E2 TTS with (P1), (P2), and (P3) configurations, respectively.

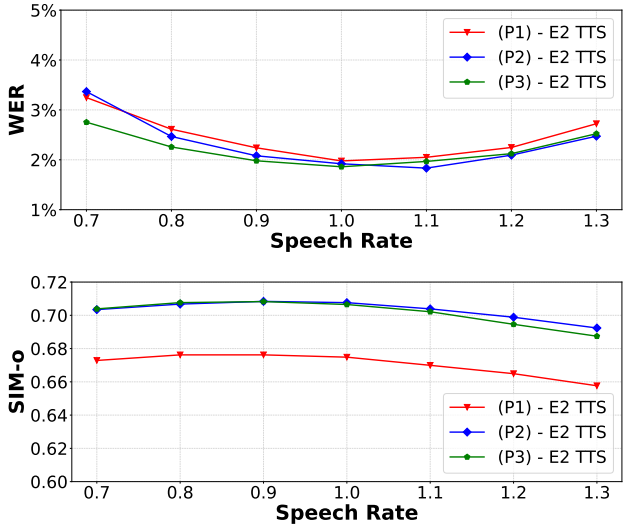


Fig. 6. WER and SIM-o of different speech rates between 0.7 to 1.3 for E2TTS with (P1), (P2), and (P3) configurations.

3.6. Analysis of the system behavior

3.6.1. Training progress

Fig. 4 illustrates the training progress of the (B4)-Voicebox, (B5)-Voicebox, (P1)-E2 TTS, and (P2)-E2 TTS models. The upper graphs represent the training progress as measured by WER, while the lower graphs depict the progress as measured by SIM-o. We present a comparison between (B4)-Voicebox and (P1)-E2 TTS, as well as between (B5)-Voicebox and (P2)-E2 TTS. The former pair was trained from scratch, while the latter was initialized by unsupervised pre-training [34].

From the WER graphs, we observe that the Voicebox models demonstrated a good WER even at the 10% training point, owing to the use of frame-wise phoneme alignment. On the other hand, E2 TTS required significantly more training to converge. Interestingly, E2 TTS achieved a better WER at the end of the training. We speculate this is because the E2 TTS model learned a more effective grapheme-to-phoneme mapping based on the large training data, compared to what was used for Voicebox.

From the SIM-o graphs, we also observed that E2 TTS required more training iteration, but it ultimately achieved a better result at the end of the training. We believe this suggests the superiority of E2 TTS, where the audio model and duration model are jointly learned as a single flow-matching Transformer.

3.6.2. Impact of audio prompt length

During the inference of E2 TTS, the model needs to automatically identify the boundary of y^{aud} and y^{text} in $\hat{y}$ based on the audio prompt $\hat{s}^{\text{aud}}$. Otherwise, the generated audio may either contain a part of y^{aud} or miss a part of y^{text}. This is not a trivial problem, and E2 TTS could show a deteriorated result when the length of the audio prompt $\hat{s}^{\text{aud}}$ is long.

To examine the capability of E2 TTS, we evaluated the WER and SIM-o with different audio prompt lengths. The result is shown in Fig. 5. In this experiment, we utilized the entire audio prompts instead of using the last 3 seconds, and categorized the result based on the length of the audio prompts. As shown in the figure, we did not observe any obvious pattern between the WER and the length of the audio prompt. This suggests that E2 TTS works reasonably well even when the prompt length is as long as 10 seconds. On the other hand, SIM-o significantly improved when the audio prompt length increased, which suggests the scalability of E2 TTS with respect to the audio prompt length.

3.6.3. Impact of changing the speech rate

We further examined the model's ability to produce suitable content when altering the total duration input. In this experiment, we adjusted the total duration by multiplying it by $\frac{1}{sr}$, where sr represents the speech rate. The results are shown in Fig. 6. As depicted in the graphs, the E2 TTS model exhibited only a moderate increase in WER while maintaining a high SIM-o, even in the challenging cases of $sr = 0.7$ and $sr = 1.3$. This result suggests the robustness of E2 TTS with respect to the total duration input.

4. CONCLUSIONS

We introduced E2 TTS, a novel fully NAR zero-shot TTS. In the E2 TTS framework, the text input is converted into a character sequence with filler tokens to match the length of the input character sequence and the output mel-filterbank sequence. The flow-matching-based mel spectrogram generator is then trained based on the audio infilling task. Despite its simplicity, E2 TTS achieved state-of-the-art zero-shot TTS capabilities that were comparable to or surpass previous works, including Voicebox and NaturalSpeech 3. The simplicity of E2 TTS also allowed for flexibility in the input representation. We proposed several variants of E2 TTS to improve usability during inference.

5. REFERENCES

[1] Yi Ren, Yangjun Ruan, Xu Tan, Tao Qin, Sheng Zhao, Zhou Zhao, and Tie-Yan Liu, "FastSpeech: Fast, robust and controllable text to speech," *Proc. NeurIPS*, vol. 32, 2019.

[2] Yi Ren, Chenxu Hu, Xu Tan, Tao Qin, Sheng Zhao, Zhou Zhao, and Tie-Yan Liu, "FastSpeech 2: Fast and high-quality end-to-end text to speech," in *Proc. ICLR*, 2021.

[3] Jaehyeon Kim, Sungwon Kim, Jungil Kong, and Sungroh Yoon, "Glow-TTS: A generative flow for text-to-speech via monotonic alignment search," *Proc. NeurIPS*, vol. 33, pp. 8067–8077, 2020.

[4] Jungil Kong, Jaehyeon Kim, and Jaekyoung Bae, "HiFi-GAN: Generative adversarial networks for efficient and high fidelity speech synthesis," *Proc. NeurIPS*, vol. 33, pp. 17022–17033, 2020.

[5] Xu Tan, Jiawei Chen, Haohe Liu, Jian Cong, Chen Zhang, Yanqing Liu, Xi Wang, Yichong Leng, Yuanhao Yi, Lei He, et al., "Naturalspeech: End-to-end text-to-speech synthesis with human-level quality," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024.

[6] Sercan Arik, Jitong Chen, Kainan Peng, Wei Ping, and Yanqi Zhou, "Neural voice cloning with a few samples," *Proc. NeurIPS*, vol. 31, 2018.

[7] Ye Jia, Yu Zhang, Ron Weiss, Quan Wang, Jonathan Shen, Fei Ren, Patrick Nguyen, Ruoming Pang, Ignacio Lopez Moreno, Yonghui Wu, et al., "Transfer learning from speaker verification to multispeaker text-to-speech synthesis," *Proc. NeurIPS*, vol. 31, 2018.

[8] Chengyi Wang, Sanyuan Chen, Yu Wu, Ziqiang Zhang, Long Zhou, Shujie Liu, Zhuo Chen, Yanqing Liu, Huaming Wang, Jinyu Li, et al., "Neural codec language models are zero-shot text to speech synthesizers," *arXiv preprint arXiv:2301.02111*, 2023.

[9] Xiaofei Wang, Manthan Thakker, Zhuo Chen, Naoyuki Kanda, Sefik Emre Eskimez, Sanyuan Chen, Min Tang, Shujie Liu, Jinyu Li, and Takuya Yoshioka, "Speechx: Neural codec language model as a versatile speech transformer," *arXiv preprint arXiv:2308.06873*, 2023.

[10] Chenpeng Du, Yiwei Guo, Hankun Wang, Yifan Yang, Zhikang Niu, Shuai Wang, Hui Zhang, Xie Chen, and Kai Yu, "Vall-t: Decoder-only generative transducer for robust and decoding-controllable text-to-speech," *arXiv preprint arXiv:2401.14321*, 2024.

[11] Detai Xin, Xu Tan, Kai Shen, Zeqian Ju, Dongchao Yang, Yuancheng Wang, Shinnosuke Takamichi, Hiroshi Saruwatari, Shujie Liu, Jinyu Li, et al., "RALL-E: Robust codec language modeling with chain-of-thought prompting for text-to-speech synthesis," *arXiv preprint arXiv:2404.03204*, 2024.

[12] Sanyuan Chen, Shujie Liu, Long Zhou, Yanqing Liu, Xu Tan, Jinyu Li, Sheng Zhao, Yao Qian, and Furu Wei, "VALL-E 2: Neural Codec Language Models are Human Parity Zero-Shot Text to Speech Synthesizers," *arXiv preprint arXiv:2402.07383*, 2024.

[13] Dongchao Yang, Jinchuan Tian, Xu Tan, Rongjie Huang, Songxiang Liu, Xuankai Chang, Jiatong Shi, Sheng Zhao, Jiang Bian, Xixin Wu, et al., "Uniaudio: An audio foundation model toward universal audio generation," *arXiv preprint arXiv:2310.00704*, 2023.

[14] Kai Shen, Zeqian Ju, Xu Tan, Yanqing Liu, Yichong Leng, Lei He, Tao Qin, Sheng Zhao, and Jiang Bian, "NaturalSpeech 2: Latent diffusion models are natural and zero-shot speech and singing synthesizers," *arXiv preprint arXiv:2304.09116*, 2023.

[15] Zeqian Ju, Yuancheng Wang, Kai Shen, Xu Tan, Detai Xin, Dongchao Yang, Yanqing Liu, Yichong Leng, Kaitao Song, Siliang Tang, et al., "Naturalspeech 3: Zero-shot speech synthesis with factorized codec and diffusion models," *arXiv preprint arXiv:2403.03100*, 2024.

[16] Jonathan Ho, Ajay Jain, and Pieter Abbeel, "Denoising diffusion probabilistic models," in *Proc. NIPS*, 2020, vol. 33, pp. 6840–6851.

[17] Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole, "Score-based generative modeling through stochastic differential equations," in *Proc. ICML*, 2020.

[18] Matthew Le, Apoorv Vyas, Bowen Shi, Brian Karrer, Leda Sari, Rashel Moritz, Mary Williamson, Vimal Manohar, Yossi Adi, Jay Mahadeokar, et al., "Voicebox: Text-guided multilingual universal speech generation at scale," *Advances in neural information processing systems*, vol. 36, 2024.

[19] Shivam Mehta, Ruibo Tu, Jonas Beskow, Éva Székely, and Gustav Eje Henter, "Matcha-TTS: A fast TTS architecture with conditional flow matching," in *Proc. ICASSP*. IEEE, 2024, pp. 11341–11345.

[20] Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le, "Flow matching for generative modeling," in *Proc. ICLR*, 2022.

[21] Vadim Popov, Ivan Vovk, Vladimir Gogoryan, Tasnima Sadekova, and Mikhail Kudinov, "Grad-TTS: A diffusion probabilistic model for text-to-speech," in *Proc. ICML*, 2021, pp. 8599–8608.

[22] Yuan Gao, Nobuyuki Morioka, Yu Zhang, and Nanxin Chen, "E3 TTS: Easy end-to-end diffusion-based text to speech," in *2023 IEEE Automatic Speech Recognition and Understanding Workshop (ASRU)*. IEEE, 2023, pp. 1–8.

[23] Olaf Ronneberger, Philipp Fischer, and Thomas Brox, "U-net: Convolutional networks for biomedical image segmentation," in *Proc. MICCAI*. Springer, 2015, pp. 234–241.

[24] Philip Anastassiou, Jiawei Chen, Jitong Chen, Yuanzhe Chen, Zhuo Chen, Ziyi Chen, Jian Cong, Lelai Deng, Chuang Ding, Lu Gao, et al., "Seed-TTS: A Family of High-Quality Versatile Speech Generation Models," *arXiv preprint arXiv:2406.02430*, 2024.

[25] Bo Li, Yu Zhang, Tara Sainath, Yonghui Wu, and William Chan, "Bytes are all you need: End-to-end multilingual speech recognition and synthesis with bytes," in *Proc. ICASSP*, 2019, pp. 5621–5625.

[26] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin, "Attention is all you need," *Proc. NIPS*, vol. 30, 2017.

[27] Ricky TQ Chen, Yulia Rubanova, Jesse Bettencourt, and David K Duvenaud, "Neural ordinary differential equations," in *Proc. NeurIPS*, 2018, vol. 31.

[28] Michael McAuliffe, Michaela Socolof, Sarah Mihuc, Michael Wagner, and Morgan Sonderegger, "Montreal forced aligner: Trainable text-speech alignment using kaldi.," in *Proc. Interspeech*, 2017, pp. 498–502.

[29] Kevin Lenzo, "The carnegie mellon university pronouncing dictionary," .

[30] Wei Kang, Xiaoyu Yang, Zengwei Yao, Fangjun Kuang, Yifan Yang, Liyong Guo, Long Lin, and Daniel Povey, "LibriHeavy: a 50,000 hours ASR corpus with punctuation casing and context," in *Proc. ICASSP*. IEEE, 2024, pp. 10991–10995.

[31] Jacob Kahn, Morgane Rivière, Weiyi Zheng, Evgeny Kharitonov, Qiantong Xu, Pierre-Emmanuel Mazaré, Julien Karadayi, Vitaliy Liptchinsky, Ronan Collobert, Christian Fuegen, et al., "Libri-light: A benchmark for ASR with limited or no supervision," in *Proc. ICASSP*, 2020, pp. 7669–7673.

[32] Sang-gil Lee, Wei Ping, Boris Ginsburg, Bryan Catanzaro, and Sungroh Yoon, "BigVGAN: A universal neural vocoder with large-scale training," in *Proc. ICLR*, 2022.

[33] Jonathan Ho and Tim Salimans, "Classifier-free diffusion guidance," *arXiv preprint arXiv:2207.12598*, 2022.

[34] Xiaofei Wang, Sefik Emre Eskimez, Manthan Thakker, Hemin Yang, Zirun Zhu, Min Tang, Yufei Xia, Jinzhu Li, Sheng Zhao, Jinyu Li, and Naoyuki Kanda, "An investigation of noise robustness for flow-matching-based zero-shot TTS," in *Proc. Interspeech*, 2024.

[35] Aleksandr Meister, Matvei Novikov, Nikolay Karpov, Evelina Bakhturina, Vitaly Lavrukhin, and Boris Ginsburg, "LibriSpeech-PC: Benchmark for evaluation of punctuation and capitalization capabilities of end-to-end asr models," in *Proc. ASRU*. IEEE, 2023, pp. 1–7.

[36] Vassil Panayotov, Guoguo Chen, Daniel Povey, and Sanjeev Khudanpur, "LibriSpeech: an ASR corpus based on public domain audio books," in *Proc. ICASSP*, 2015, pp. 5206–5210.

[37] Wei-Ning Hsu, Benjamin Bolte, Yao-Hung Hubert Tsai, Kushal Lakhotia, Ruslan Salakhutdinov, and Abdelrahman Mohamed, "HuBERT: Self-supervised speech representation learning by masked prediction of hidden units," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 29, pp. 3451–3460, 2021.

[38] Sanyuan Chen, Chengyi Wang, Zhengyang Chen, Yu Wu, Shujie Liu, Zhuo Chen, Jinyu Li, Naoyuki Kanda, Takuya Yoshioka, Xiong Xiao, et al., "WavLM: Large-scale self-supervised pre-training for full stack speech processing," *IEEE Journal of Selected Topics in Signal Processing*, vol. 16, no. 6, pp. 1505–1518, 2022.